

CLI: main_chatbot

Terminal-Chatbot auf Basis der Provider-Schicht

Zweck

main_chatbot.py liefert einen zustandsbehafteten Terminal-Chatbot mit Verlauf, Streaming und Reset. Das Modul ist bewusst eigenständig: Es nutzt ausschließlich die öffentliche Schnittstelle der Konfigurations- und Provider-Schicht — konkret rag_project.llm.providers — und ändert keine andere Datei des Projekts.

Achtung bei älteren Beschreibungen: Verweise auf rag_project.config und rag_project.providers sind veraltet. Beide Module liegen heute unter core/ beziehungsweise llm/.

Aufruf

Der Einstieg ist als Skript in pyproject.toml eingetragen und heißt chatbot.

- `uv run chatbot` — REPL mit Provider und Modell aus der .env
- `uv run chatbot --no-stream` — Antworten vollständig statt stückweise
- `uv run chatbot --model openrouter:z-ai/glm-5.3` — Modell für diesen Lauf erzwingen

Die Klasse ChatBot

Der Chatbot hält seinen Verlauf als LangChain-Nachrichtenliste: die System-Nachricht an Position 0, danach abwechselnd Mensch und Modell. Der Client wird mit streaming=True gebaut — auch dann, wenn die REPL gerade im Modus invoke arbeitet. So muss der Client nur einmal einen Stream beherrschen.

Element	Art	Wirkung
system_prompt	Feld	aktueller System-Prompt, Standard „You are a friendly Python tutor.“
model	Feld	optionale Modellkennung, schlägt CHAT_MODEL
temperature	Feld	optionale Temperatur; ist sie None, wird der Parameter gar nicht gesetzt
invoke(message)	Methode	Anfrage senden, vollständige Antwort erhalten

Element	Art	Wirkung
stream(message)	Methode	Textstücke ausgeben, sobald sie eintreffen
reset(prompt)	Methode	Verlauf leeren, System-Prompt optional ersetzen
model_label	Eigenschaft	Kennung, Adresse, Temperatur und max_tokens als Zeile
turn_count()	Methode	Zahl der Nutzer-Turns ohne System-Nachricht

Verlauf und Streaming

invoke() hängt die Frage an den Verlauf, ruft den Client und hängt die Antwort an. stream() geht anders vor: Es sammelt die Stücke, gibt sie sofort aus und ergänzt den Verlauf erst nach dem letzten Stück mit der zusammengesetzten Antwort. Das ist Absicht — ein abgebrochener Stream soll keine halbe Antwort im Verlauf hinterlassen.

Die Hilfsfunktion _as_text() reduziert die Antwort auf reinen Text. Je nach Anbieter ist der Inhalt ein String oder eine Liste von Blöcken; beides wird hier auf Text gebracht, wobei Textblöcke aneinandergefügt werden.

Reset und System-Prompt

reset() baut den Verlauf auf eine einzelne System-Nachricht zurück. Wird ein Prompt übergeben, ersetzt er den bisherigen — und der Client wird neu gebaut. Ein leerer oder nur aus Leerzeichen bestehender Prompt stellt den Standard wieder her. Dass der Client beim Reset neu entsteht, ist getestet: Die Suite erwartet genau zwei Aufrufe von build_llm, jeweils mit streaming=True.

Die REPL

Die Schleife liest eine Zeile, verarbeitet sie und trennt dabei nach Präfix: Alles, was mit einem Schrägstrich beginnt, ist ein Befehl; alles andere geht an den Chatbot. Unbekannte Befehle werden gemeldet, ohne die Schleife zu beenden. Strg-C und Dateiende beenden sie sauber.

Befehl	Wirkung
/help	Übersicht der Befehle
/system <Text>	System-Prompt ersetzen und Verlauf leeren
/system	Standard-Prompt wiederherstellen
/show	aktuellen System-Prompt zeigen

Befehl	Wirkung
/reset	Verlauf leeren, System-Prompt behalten
/model	aktives Modell und Provider anzeigen
/mode <stream invoke>	Ausgabemodus wechseln
/exit oder /quit	beenden

Bei /system bleibt die Groß- und Kleinschreibung des Prompts erhalten, weil der Text aus der unveränderten Eingabezeile gelesen wird. Die Befehlsauswertung selbst arbeitet mit kleingeschriebenen Schlüsselwörtern.

Der Modus startet abhängig von --no-stream als stream oder invoke und lässt sich zur Laufzeit mit /mode umschalten. Beim Wechsel nach invoke weist die REPL darauf hin, dass Antworten nun vollständig statt stückweise eintreffen.

Fehlerverhalten

Netz-, Auth- und API-Fehler werden je Runde abgefangen, als [Fehler] nach stderr geschrieben und die Schleife läuft weiter. Ein Strg-C während einer Antwort wird als [abgebrochen] gemeldet, ohne den Verlauf zu beschädigen.

Testbezug

tests/test_main_chatbot.py ersetzt build_llm durch eine Attrappe und prüft, dass der Client beim Aufbau und beim System-Reset genau einmal neu gebaut wird — beide Male mit streaming=True. Der Test läuft ohne Netz und ohne Schlüssel.